

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №4
по дисциплине: **Операционные системы**
тема: **«Разработка драйвера для ОС Linux»**

Выполнил: ст. группы ВТ-231

Масленников Д. А.

Проверили:
Островский А. М.

Белгород 2025 г

Лабораторная работа № 4

Цель работы: Изучить основы разработки драйвера для ОС Linux с использованием языка программирования C, включая настройку окружения, реализацию и тестирование.

Цель работы обуславливает постановку и решение следующих задач:

1. Настроить окружение для разработки драйвера на языке C для ОС Linux.
2. Разработать и протестировать демо-драйвер `tu_sum` на языке C.
 - 2.1. Освоить основные принципы программирования драйвера.
 - 2.2. Реализовать работу с мьютексами в драйвере.
3. Выполнить индивидуальное задание, закрепляющее на практике полученные знания (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером). Задание должно быть выполнено, либо на языке C, либо на языке Rust.
 - 3.1. Добавить логирование операций с использованием встроенных средств ядра Linux.
 - 3.2. Создать структуру для управления ресурсами драйвера, включая выделение и освобождение памяти.
 - 3.3. Протестировать работу разработанного драйвера на производительность при высоком числе одновременных запросов.
4. Составить отчет, включающий код разработанного драйвера, результаты тестирования и выводы.

Вариант 14

14. Разработать драйвер-эмулятор для устройства "виртуальная батарея". Драйвер должен предоставлять интерфейс `/dev/virtual_battery`, в котором отображается уровень заряда батареи (от 0% до 100%) и состояние зарядки (заряжается или разряжается). Уровень заряда изменяется каждые 5 секунд в зависимости от текущего состояния

Выполнение индивидуального задания:

Код программы на языке программирования Rust:

```
use std::fs;
use std::thread;
use std::time::{Duration, SystemTime, UNIX_EPOCH};

// Конфигурация драйвера
const UPDATE_INTERVAL: u64 = 1;
const BAT_PATH: &str = "/sys/class/power_supply/BAT0";

// Цветовая палитра
const RESET: &str = "\x1b[0m";
const BOLD: &str = "\x1b[1m";
const RED: &str = "\x1b[31m";
const GREEN: &str = "\x1b[32m";
const YELLOW: &str = "\x1b[33m";
const BLUE: &str = "\x1b[34m";
const CYAN: &str = "\x1b[36m";

struct BatteryDriver {
    sysfs_path: String,
}

impl BatteryDriver {
    // Функция инициализации (Probe)
    fn new(path: &str) -> Self {
        Self {
            sysfs_path: path.to_string(),
        }
    }

    // Внутренний метод чтения "регистра" (файла)
    fn read_file(&self, file: &str) -> String {
        let path = format!("{}/{}", self.sysfs_path, file);
        fs::read_to_string(path)
            .unwrap_or_else(|_| "Unknown".to_string())
            .trim()
            .to_string()
    }

    // API драйвера: Получение уровня заряда
    fn get_capacity(&self) -> u8 {
        self.read_file("capacity")
            .parse()
            .unwrap_or(0)
    }

    // API драйвера: Получение статуса
```

```

fn get_status(&self) -> String {
    self.read_file("status")
}

// API драйвера: Получение профиля питания (CPU)
// Это виртуальное устройство, агрегирующее данные из другой подсистемы
fn get_power_profile(&self) -> (String, String) {
    if let Ok(epp) = fs::read_to_string("/sys/devices/system/cpu/cpu0/cpufreq/energy_performance_preference") {
        let e = epp.trim();
        match e {
            "performance" => (format!("Максимальная производительность"), RED.to_string()),
            "balance_performance" => (format!("Сбалансированный"), BLUE.to_string()),
            "power" => (format!("Энергосберегающий"), GREEN.to_string()),
            _ => (format!("⚙️ {}", e), RESET.to_string()),
        }
    } else {
        ("Ошибка чтения профиля".to_string(), RESET.to_string())
    }
}

fn draw_progress_bar(level: u8, is_charging: bool) -> String {
    let width = 50;
    let filled_len = (level as f32 / 100.0 * width as f32).round() as usize;
    let empty_len = width - filled_len;

    let color = if is_charging {
        CYAN
    } else if level > 60 {
        GREEN
    } else if level > 25 {
        YELLOW
    } else {
        RED
    };

    let filled: String = std::iter::repeat("█").take(filled_len).collect();
    let empty: String = std::iter::repeat("░").take(empty_len).collect();

    format!("{}", color, filled, RESET, empty)
}

fn fetch_state_snapshot(&self) {
    let capacity = self.get_capacity();
    let status = self.get_status();
    let (profile_name, profile_color) = self.get_power_profile();
    let is_charging = status == "Charging";

    println!("\x1Bc");
    println!("Время системы: {}", get_timestamp());
}

```

```

println!("\n{}Информация о батарее{}", BOLD, RESET);

let status_icon = if is_charging { "Подключено питание" } else { "Работа от батареи" };
println!("Статус: {}{}{}", BOLD, status_icon, RESET);

let bar = Self::draw_progress_bar(capacity, is_charging);
println!("Заряд: {}{}%{}", BOLD, capacity, RESET);
println!("{}", bar);

println!("\n{}Режим CPU:{}", BOLD, RESET);
println!("Profile: {}{}{}", profile_color, profile_name, RESET);
}
}

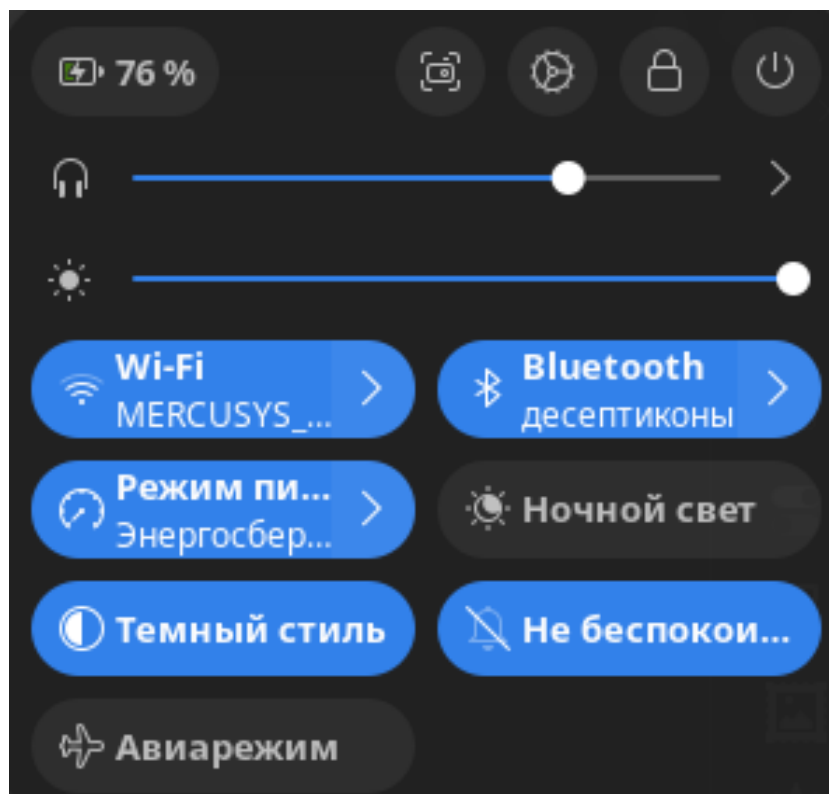
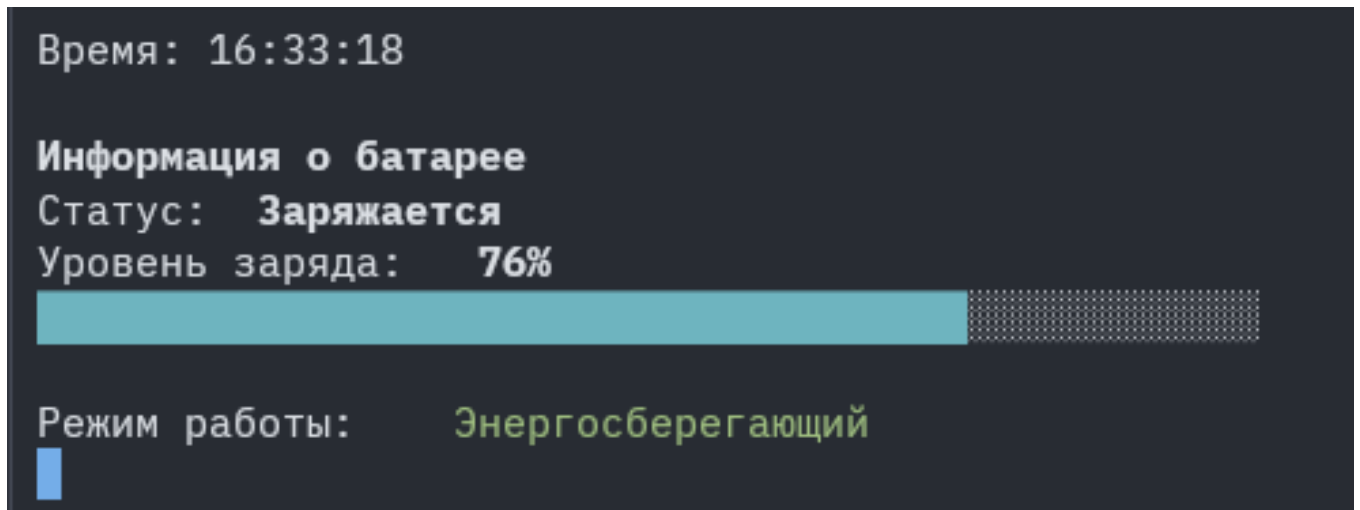
// Вспомогательная функция времени
fn get_timestamp() -> String {
    let now = SystemTime::now().duration_since(UNIX_EPOCH).unwrap().as_secs();
    let seconds = now % 60;
    let minutes = (now / 60) % 60;
    let hours = (now / 3600) % 24; // UTC время (для простоты)
    format!("{:02}:{:02}:{:02}", hours, minutes, seconds)
}

fn main() {
    // 1. Инициализация драйвера (Probe)
    let driver = BatteryDriver::new(BAT_PATH);

    // 2. Бесконечный цикл опроса устройства
    loop {
        driver.fetch_state_snapshot();
        thread::sleep(Duration::from_secs(UPDATE_INTERVAL));
    }
}

```

Результат работы программы:



На скриншотах видно, что программа корректно отображает процент заряда, заряжается или нет наша батарея и режим работы

Вывод: Разработан userspace-драйвер на языке Rust, инкапсулирующий взаимодействие с подсистемой питания Linux (sysfs) в структуру BatteryDriver. Реализован механизм периодического опроса состояния оборудования и визуализация параметров энергопотребления с поддержкой цветовой индикации режимов CPU.